Desafio Técnico – Backend

🎯 Objetivo

Criar uma API simples para gestão de **solicitações de antecipação de valores**, com foco em clareza de código, boas práticas de engenharia, testes automatizados e estrutura pensada para evoluir.

📚 Contexto

A LastLink permite que criadores recebam suas receitas por meio da plataforma. Para ajudar no fluxo de caixa, disponibilizamos a opção de **antecipação**: o criador pode solicitar que parte de seus recebíveis futuros seja liberada antes do prazo, mediante uma taxa.

Você foi convidado a construir um **serviço de antecipação de valores**. Esse serviço será consumido por um sistema interno e precisa expor uma API REST para gerenciar as solicitações.

🧱 Requisitos

📌 Funcionalidades da API

Criar uma solicitação de antecipação(creator)

Informar: creator_id, valor_solicitado, data_solicitacao

Aplicar taxa de 5% sobre o valor solicitado

Calcular e retornar: valor_liquido, status (default = “pendente”)

Listar solicitações por creator_id(todos)

Aprovar ou recusar uma solicitação(Admin)

Atualizar status: “aprovada” ou “recusada”

(Opcional) Expor endpoint para simulação sem criar solicitação (GET com query params)

🔎 Regras de negócio

Valor solicitado deve ser maior que R$ 100,00

Um creator não pode ter mais de uma solicitação pendente ao mesmo tempo

A taxa de antecipação é fixa: 5% sobre o valor bruto

Toda solicitação inicia com status **pendentef**

🧪 O que esperamos ver

Código claro, organizado, coeso

Testes automatizados (unitários ou de integração)

Modelagem bem pensada (domínio separado de controller, por exemplo)

Uso de versionamento de API (v1)

Um README explicando como rodar localmente (sem dor)

(Opcional) 🌐 Frontend

Se quiser mostrar skills fullstack, você pode entregar uma tela simples em Angular ou React que:

Mostre uma lista de solicitações existentes

Permita criar uma nova solicitação com formulário

Permita aprovar/reprovar

🧰 Stack sugerida

Backend: C# / .NET Core (ou stack à sua escolha)

Banco de dados: em memória (ex: SQLite ou mock em memória)

Testes: framework da sua stack preferida

🚀 Entrega

Suba em um repositório no GitHub/GitLab ou envie um .zip

README com instruções de execução e testes

Caso use Postman, inclua uma collection

Prazo sugerido: 5 dias úteis

🎯 Objetivo

Projetar e implementar uma **API para gestão de solicitações de antecipação de valores**, com foco em:

- clareza e organização do código
- boas práticas de engenharia

- modelagem de domínio
- testes automatizados
- decisões arquiteturais bem justificadas

Mais importante do que a complexidade da solução é **como você estrutura o código e explica suas decisões técnicas**.

📚 Contexto

A **LastLink** permite que criadores monetizem conteúdo e recebam pagamentos pela plataforma.

Os valores gerados pelos creators ficam disponíveis para saque conforme regras de liquidação. Para ajudar no fluxo de caixa, a plataforma oferece **antecipação de recebíveis**, permitindo que um creator receba parte do valor antes da liquidação, mediante uma taxa.

Este serviço será responsável por gerenciar **solicitações de antecipação**.

Existem dois perfis de usuário no sistema:

Creator

- pode **criar solicitações de antecipação**
- pode **visualizar apenas as solicitações que ele criou**

Admin

- pode **visualizar todas as solicitações**
- pode **aprovar ou recusar solicitações**

🧱 Escopo do Serviço

Você deve construir um **serviço HTTP (API REST)** responsável por gerenciar o fluxo de antecipações.

A API deve ser organizada de forma que o sistema possa **evoluir facilmente no futuro**.

📌 Funcionalidades da API

Criar solicitação de antecipação (Creator)

Criar uma nova solicitação informando:

```
creator_id
valor_solicitado
data_solicitacao
```

A API deve:

- aplicar **taxa de 5%** sobre o valor solicitado
- calcular o **valor líquido**
- criar a solicitação com status inicial **PENDENTE**

Exemplo de resposta:

```
{
  "id": "uuid",
  "creator_id": "123",
  "valor_bruto": 1000,
  "taxa": 50,
  "valor_liquido": 950,
  "status": "pendente",
  "created_at": "2026-01-10T10:00:00"
}
```

Listar solicitações

Todos os usuários podem listar solicitações, porém com regras de acesso:

Creator

- pode listar **apenas as solicitações criadas por ele**

Admin

- pode listar **todas as solicitações**

Opcionalmente, a API pode suportar filtros como:

- status
- data
- creator_id

Aprovar ou recusar solicitação (Admin)

Permitir atualizar o status de uma solicitação para:

- **APROVADA**
- **RECUSADA**

(Opcional) Simular antecipação

Criar um endpoint que permita **simular uma antecipação sem criar a solicitação**.

Exemplo:

```
GET /v1/antecipacoes/simulacao?valor=1000
```

Retornando:

```
valor_bruto
taxa
valor_liquido
```

🔎 Regras de Negócio

- o **valor mínimo da solicitação é R$ 100**
- um **creator não pode ter mais de uma solicitação pendente ao mesmo tempo**
- a **taxa de antecipação é fixa: 5%**
- toda solicitação inicia com status **PENDENTE**

⚠️ Concorrência

Considere que múltiplas requisições podem ocorrer **simultaneamente** para o mesmo creator.

O sistema deve garantir que **um creator nunca tenha mais de uma solicitação pendente ao mesmo tempo**, mesmo sob concorrência.

Você pode resolver isso da forma que considerar mais adequada.

Caso não implemente essa proteção, descreva no **README** como resolveria em um ambiente de produção.

📜 Auditoria

Mudanças de status de uma solicitação devem ser **auditáveis**.

Descreva no README como você garantiria:

- histórico de alterações de status
- rastreabilidade de decisões
- integridade das informações

A implementação é opcional, mas a abordagem deve ser explicada.

📈 Evolução do Sistema

Considere que futuramente a plataforma poderá:

- ter **milhares de creators solicitando antecipação simultaneamente**
- suportar **diferentes taxas de antecipação**
- integrar com **sistemas financeiros externos**

Descreva no **README** como o sistema poderia evoluir para lidar com esses cenários.

🧪 O que esperamos ver

- código claro, organizado e coeso
- separação de responsabilidades
- modelagem de domínio bem definida

- testes automatizados (unitários ou integração)
- uso de **versionamento de API (v1)**

🧰 Stack sugerida

Você pode utilizar a stack que preferir.

Sugestões:

Backend

- C# / .NET

Banco de dados

- SQLite
- Postgres
- ou banco em memória

Testes

- framework de testes da stack escolhida

🌐 (Opcional) Interface

Se quiser demonstrar habilidades **fullstack**, você pode incluir uma interface simples que permita:

- listar solicitações
- criar nova solicitação
- aprovar ou recusar solicitações

🚀 Entrega

- subir o projeto em um **repositório GitHub ou GitLab** ou enviar um `.zip`
- incluir um **README com instruções claras para execução**
- incluir **testes automatizados**
- caso utilize Postman, incluir uma **collection**

Prazo sugerido: 5 dias úteis.